ML TRAINING PROGRAM · FINAL ML ASSIGNMENT

Assignment 03

Hyperparameter Tuning, Model Persistence & Production-Ready Inference

on the UCI Adult Income Dataset

Level	Duration	Dataset	Bridge
Intermediate+	**2 Days · 8 hrs/day**	**Adult Income (Kaggle)**	**A03 → Django A04**

Kaggle: *kaggle.com/datasets/uciml/adult-census-income*

Final assignment in the ML series — this one matters beyond the notebook

Assignments 01 and 02 built skills. Assignment 03 produces a deliverable. The inference script you write at the end of Day 2 is not an exercise — it is the exact artifact that Assignment 04 (Django REST Framework) will import and wrap into a live API endpoint. Write it like it will be used in production, because it will be.

Two new concepts introduced:

- Hyperparameter tuning — RandomizedSearchCV over your existing Pipeline. You'll learn why random search beats grid search at scale, how to define a search space, and how to interpret the results.
- Model persistence — joblib serialisation of a fitted Pipeline. The saved artifact carries preprocessing + model in a single file. No separate scaler to save, no separate encoder — the Pipeline handles it.

Concepts carried forward (still tested):

- Pipeline + ColumnTransformer — same structure as A02, reused here with a different dataset.
- StratifiedKFold + cross_validate — all scores computed, none hardcoded.
- Class imbalance — same problem, different dataset, same toolkit.
- Self-directed analysis — mandatory, minimum depth enforced.

1. Overview & Context

The UCI Adult Income dataset contains demographic and employment data from the 1994 US Census. The task is binary classification: predict whether a person's income exceeds $50,000 per year based on features such as age, education, occupation, marital status, and hours worked per week.

This dataset introduces a cleaning pattern you haven't handled before: missing values encoded as the string '?' rather than NaN. A naive isnull().sum() will show zero missing values — they are invisible until you look for them. This is extremely common in real-world data exports from legacy systems.

Beyond cleaning, this assignment is about optimisation and delivery. You've built pipelines. You've evaluated models. Now you tune the best one and ship it — as a serialised artifact with a clean inference interface that another engineer (or your Assignment 04 self) can load and call without seeing the training code.

Rows	48,842 (train + test combined; split yourself)
Features	14 (numeric + categorical, mixed)
Target	income: >50K / <=50K (binary classification)
Class split	~76% <=50K / ~24% >50K — imbalanced
Key trap	'?' used as missing marker instead of NaN — invisible to isnull()
New: Tuning	RandomizedSearchCV over Pipeline hyperparameters
New: Persist	joblib.dump() — save full pipeline as .pkl artifact
A04 bridge	inference.py — standalone prediction function imported by Django

2. Learning Objectives

By the end of this assignment you will be able to:

- Detect and handle missing values encoded as strings ('?') rather than NaN — a pattern common in legacy data exports.
- Define a hyperparameter search space over a Pipeline using the double-underscore notation (classifier__n_estimators, preprocessor__num__imputer__strategy).
- Choose between GridSearchCV and RandomizedSearchCV based on search space size and time constraints, and justify the choice in writing.
- Run a RandomizedSearchCV, extract the best estimator, and evaluate it against pre-tuning baseline on the same CV split.
- Serialise a fully-fitted Pipeline (preprocessor + classifier) to disk using joblib.dump().
- Write a production-ready inference.py script: load the artifact, validate input schema, predict, return structured output.
- Document the expected input schema and output contract of the inference function — the foundation of an API spec.
- Conduct and present a genuine self-directed analysis with minimum depth requirements.

3. Two-Day Schedule

Total: 16 hours · 8 hours per day. Day 1 is data + pipeline. Day 2 is tuning + persistence + inference script. The inference script is the most important output — do not sacrifice it to polish earlier sections.

Day 1 — Data Audit, Cleaning, Baseline Pipeline

Goal for Day 1: A fitted baseline Pipeline with computed CV scores. Clean data with no '?' values remaining. Feature engineering documented.

Block 1 · Hours 1–2 · Audit — Find the Hidden Missing Values

This is the key cleaning challenge in this dataset. Approach it carefully.

- Load adult.csv. Print shape, dtypes, value_counts() on the income column.
- Run df.isnull().sum() — you will see zero missing values. This is misleading.
- Now run: df.isin(['?']).sum() — you will find missing values in workclass, occupation, and native.country encoded as the string '?'.
- Replace all '?' with np.nan: df.replace('?', np.nan, inplace=True). Rerun isnull().sum() to confirm the real missing count.
- income column has values ' >50K' and ' <=50K' — note the leading space. Strip whitespace from all string columns before any encoding: df[col] = df[col].str.strip() for all object columns.
- Encode target: income '>50K' → 1, '<=50K' → 0. Store as y.
- Print class distribution as counts and percentages. Visualise as bar chart.
- Write a markdown audit summary: what hidden issues you found, what you fixed, why.

Block 2 · Hours 3–4 · Feature Engineering

All features engineered before the pipeline is built. Document every decision.

- Create age_group: bin age into Young (18-30), Mid-Career (31-45), Senior (46-60), Pre-Retirement (61+) as a categorical feature.
- Create hours_category: bin hours.per.week into Part-Time (<35), Full-Time (35-45), Overtime (46-60), Extreme (>60).
- Create capital_net: capital.gain - capital.loss. Many rows have 0 for both — this single feature captures financial activity more cleanly than two sparse columns.
- Create is_married: binary flag — 1 if marital.status contains 'Married', 0 otherwise. Reduces the 7-category marital.status to a strong binary signal.
- Create education_level: map education to a numeric ordinal (Preschool=1 … Doctorate=16). Use the natural academic ladder. Preserves ordinality that one-hot encoding would lose.

- Document each feature in a markdown cell: what it is, what signal you expect it to carry.

Block 3 · Hours 5–6 · Pipeline Construction

Same architecture as Assignment 02. Apply it here without the starter scaffold — you have done this before.

- Identify numeric and categorical columns from X_train (after split). Add the coverage assert from Assignment 02 feedback.
- Build numeric_transformer: SimpleImputer(strategy='median') → StandardScaler().
- Build categorical_transformer: SimpleImputer(strategy='most_frequent') → OneHotEncoder(handle_unknown='ignore', drop='first').
- Build ColumnTransformer with remainder='drop'. Verify with the coverage assert.
- Build baseline Pipeline with LogisticRegression(class_weight='balanced', max_iter=1000).
- Fit on X_train. Evaluate on X_val using classification_report + confusion matrix + AUC.

Block 4 · Hours 7–8 · Baseline Comparison & Imbalance

- Train DummyClassifier baseline. Record accuracy. Note that it will predict <=50K for every row — 76% accuracy, zero recall on the >50K class.
- Train all 4 models from A02 using the same evaluate_pipeline() pattern. Use StratifiedKFold(n_splits=5). All scores computed via cross_validate.
- Build the results table. Sort by CV AUC. Identify your best baseline model — this is the model you will tune on Day 2.
- Save a markdown note: which model won, by how much, what the Train/CV gap looks like for each.

Day 2 — Tuning, Persistence & Inference Script

Goal for Day 2: A tuned model with measurably improved CV AUC. A saved .pkl pipeline. A standalone inference.py that loads and calls the artifact cleanly.

Block 5 · Hours 1–3 · Hyperparameter Tuning with RandomizedSearchCV

GridSearch vs RandomizedSearch — read this before writing any code

GridSearchCV exhaustively tries every combination in the search space. With 4 hyperparameters each having 4 values, that is 4^4 = 256 fits × 5 folds = 1,280 model fits. On a 40K row dataset with RandomForest, this can take hours.

RandomizedSearchCV samples n_iter random combinations from the space. With n_iter=30, you get 30 × 5 = 150 fits — about 12% of the work — and consistently find near-optimal solutions because most of the performance gain comes from finding the right order of magnitude

for each parameter, not the exact value. Use RandomizedSearchCV whenever your search space has more than ~20 total combinations.

- Take your best baseline model from Block 4. Define a param_distributions dict for RandomizedSearchCV. Use the double-underscore pipeline notation.

Reference param_distributions structure (fill in the values yourself — do not copy blindly):

```
# For RandomForest best model — adjust for your actual best model
param_distributions = {
    'classifier__n_estimators':      [100, 200, 300, 500],
    'classifier__max_depth':          [None, 5, 10, 15, 20],
    'classifier__min_samples_leaf':   [1, 2, 5, 10],
    'classifier__max_features':       ['sqrt', 'log2', 0.3, 0.5],
    'preprocessor__num__imputer__strategy': ['median', 'mean'],
}

# If best model is LogisticRegression:
# param_distributions = {
#     'classifier__C':         [0.001, 0.01, 0.1, 1, 10, 100],
#     'classifier__penalty':    ['l1', 'l2'],
#     'classifier__solver':     ['liblinear', 'saga'],
# }
```

- Define RandomizedSearchCV with n_iter=30, cv=skf, scoring='roc_auc', n_jobs=-1, random_state=42, refit=True.
- Fit on X_train. This will take several minutes. Do not interrupt it.
- After fitting: print search.best_params_ and search.best_score_. Compare best_score_ to your pre-tuning baseline CV AUC from Block 4.
- Extract search.best_estimator_ — this is your tuned Pipeline.
- Evaluate the tuned pipeline on X_val: print classification_report, AUC, confusion matrix.
- Build a comparison table: Baseline CV AUC vs Tuned CV AUC vs Validation AUC. All three numbers computed, not typed.

What to do if tuning did not improve AUC

If tuned CV AUC ≤ baseline CV AUC: do not discard the exercise. Instead, investigate: Was the search space too narrow? Was n_iter too low? Was the baseline already near-optimal? Write a markdown cell explaining your diagnosis. This is valid engineering — knowing when tuning doesn't help is as important as knowing when it does.

Block 6 · Hours 4–5 · Model Persistence with joblib

What joblib saves and why it matters

joblib.dump() serialises the entire Python object — in this case, a fitted sklearn Pipeline. That includes the fitted StandardScaler (with its mean/variance), the fitted OneHotEncoder (with its

known categories), and the fitted classifier (with its weights). One file, one load call, everything needed to transform raw input and produce a prediction.

- Save the tuned pipeline to disk:

```
import joblib
import os

os.makedirs('artifacts', exist_ok=True)
joblib.dump(search.best_estimator_, 'artifacts/income_pipeline.pkl')
print('Saved. Size:', os.path.getsize('artifacts/income_pipeline.pkl'),
'bytes')
```

- Verify the artifact loads and produces identical predictions:

```
loaded_pipeline = joblib.load('artifacts/income_pipeline.pkl')

# Predictions must match exactly
original_preds = search.best_estimator_.predict(X_val)
loaded_preds   = loaded_pipeline.predict(X_val)

assert (original_preds == loaded_preds).all(), 'Loaded pipeline predictions
differ!'
print('Artifact verified. Predictions match.')
```

- Print the artifact's feature input schema — the exact column names and dtypes the pipeline expects. This becomes your API input documentation.

```
print('Expected input columns:')
for col in X_train.columns:
    print(f'  {col}: {X_train[col].dtype}')
```

Block 7 · Hours 6–7 · Write inference.py

This is the most important output of Assignment 03

inference.py is a standalone Python script that Assignment 04 will import directly into a Django view. It must work correctly when called from outside the notebook. That means: no notebook-specific globals, no reliance on variables defined elsewhere, clean imports, explicit input validation, structured output. Write it in a separate .py file, not as a notebook cell.

Required structure — implement every section:

```
# inference.py
# ------------
# Standalone inference module for the Adult Income classifier.
```

```
# Loaded by Assignment 04 Django view.

import joblib
import pandas as pd
import numpy as np
from pathlib import Path

# ── Constants ──────────────────────────────────────────────
MODEL_PATH = Path(__file__).parent / 'artifacts' / 'income_pipeline.pkl'

REQUIRED_COLUMNS = [
    'age', 'workclass', 'fnlwgt', 'education', 'educational.num',
    'marital.status', 'occupation', 'relationship', 'race', 'gender',
    'capital.gain', 'capital.loss', 'hours.per.week', 'native.country',
    # add your engineered features here
]

INCOME_LABELS = {0: '<=50K', 1: '>50K'}

# ── Load model once at import time ─────────────────────────
_pipeline = None

def _load_pipeline():
    global _pipeline
    if _pipeline is None:
        _pipeline = joblib.load(MODEL_PATH)
    return _pipeline

# ── Input validation ───────────────────────────────────────
def validate_input(data: dict) -> pd.DataFrame:
    """
    Validates and converts a raw dict to a DataFrame the pipeline accepts.
    Raises ValueError with a descriptive message on bad input.
    """
    missing = [c for c in REQUIRED_COLUMNS if c not in data]
    if missing:
        raise ValueError(f'Missing required fields: {missing}')

    df = pd.DataFrame([data])

    # Apply same string cleaning as training
    for col in df.select_dtypes(include='object').columns:
        df[col] = df[col].str.strip()

    # Apply same feature engineering as training
    # ... (your engineering functions here)

    return df[REQUIRED_COLUMNS]  # enforce column order

# ── Public predict function ────────────────────────────────
def predict(data: dict) -> dict:
    """
    Takes a raw input dict, returns prediction dict.
```

```
    Args:
        data: dict with keys matching REQUIRED_COLUMNS

    Returns:
        {
            'prediction': int (0 or 1),
            'label': str ('<=50K' or '>50K'),
            'probability': float (probability of >50K class)
        }

    Raises:
        ValueError: if required fields are missing or invalid
    """
    pipeline = _load_pipeline()
    df = validate_input(data)

    prediction  = int(pipeline.predict(df)[0])
    probability = float(pipeline.predict_proba(df)[0][1])

    return {
        'prediction':  prediction,
        'label':       INCOME_LABELS[prediction],
        'probability': round(probability, 4),
    }
```

- Test inference.py by calling predict() with at least 3 sample inputs from your test set in a notebook cell. Confirm the output dict matches your expectations.
- Test the error case: call predict() with a missing required field. Confirm it raises ValueError with a useful message.
- Print the round-trip time: load the artifact and call predict() on a single row. This is your baseline API latency.

Block 8 · Hour 8 · Self-Directed Analysis

Mandatory. Minimum requirements enforced.

Pick ONE option below. Minimum 3 original charts, 5 specific findings with numbers. Generic statements ('education affects income') do not count as findings. A finding is: 'Doctorate holders earn >50K at 74.3% — the highest rate of any education level, nearly 3× the rate of high school graduates.'

- Option A — Education & Occupation Matrix: What is the income rate (% earning >50K) for every (education, occupation) combination? Which 5 combinations have the highest rate? Which 5 the lowest? Is advanced education equally predictive across all occupations?
- Option B — Hours & Capital Analysis: How does income vary across the hours_category bins? Is the Overtime group materially better off than Full-Time? Does capital_net have a threshold effect — is there a value above which >50K becomes the majority class?

- Option C — Demographic Segment Deep Dive: Compare income rates across gender and race. Compute the income gap (% >50K: Male minus Female) within each occupation category. Which occupations have the smallest and largest gaps?

- After analysis: browse 3 Kaggle notebooks on this dataset. Find one technique not used here. Implement it. Cite all 3 with real URLs in a ## References section.

4. Bridge to Assignment 04 — What Comes Next

Assignment 04 is a Django REST Framework project. You will build a REST API that exposes your trained model as an HTTP endpoint. Here is exactly how your A03 outputs map to A04 work — understanding this should inform how you write inference.py.

A03 output → A04 usage

`artifacts/income_pipeline.pkl` → Loaded once in Django app startup via AppConfig.ready(). Never loaded per-request.

`inference.predict(data)` → Called inside a Django view function after deserialising the POST request body.

`REQUIRED_COLUMNS list` → Drives the Django REST serializer field definitions. One column = one serializer field.

`validate_input() ValueError` → Caught in the view and returned as HTTP 400 with the error message as JSON.

`predict() return dict` → Returned directly as JSON response body from the HTTP 200 response.

The A04 view will look roughly like this — you don't need to write it now, but design your inference.py with this in mind:

```
# A04 preview — what your inference.py will plug into
# (Django REST Framework view — you will write this in A04)

from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from inference import predict          # ← your A03 inference.py

class IncomePredictView(APIView):
    def post(self, request):
        try:
            result = predict(request.data)   # ← your predict() function
            return Response(result, status=status.HTTP_200_OK)
        except ValueError as e:
            return Response({'error': str(e)},
status=status.HTTP_400_BAD_REQUEST)
```

If your predict() function takes a dict and returns a dict with 'prediction', 'label', and 'probability', the view above will work with zero changes. That is the contract. Write to it.

5. Deliverables

Submit all items. D5 (inference.py) is weighted most heavily — it is the bridge to A04.

#	Deliverable	Description
D1	**Jupyter Notebook (.ipynb)**	Runs top-to-bottom. All cell outputs visible. No hardcoded CV scores anywhere.
D2	**Baseline Results Table**	Computed via evaluate_pipeline() for all 4 models. CV AUC, CV F1, CV Recall, Train AUC all present.
D3	**Tuning Comparison Table**	Baseline CV AUC vs Tuned CV AUC vs Validation AUC. All 3 values computed, not typed. search.best_params_ printed.
D4	**artifacts/income_pipeline.pkl**	Saved via joblib.dump(). Verified with the round-trip assert. File size printed.
D5	**inference.py**	Standalone script with load_pipeline(), validate_input(), predict(). Tested with 3 sample inputs and 1 error case. Latency printed.
D6	**Self-Directed Analysis**	Chosen option with ≥3 original charts and ≥5 specific numbered findings.
D7	**## References**	3 real Kaggle notebook URLs. Borrowed technique implemented and assessed.
D8	**Written Summary**	Final markdown cell: model choice, tuning gain/loss, what you'd change, one sentence on what A04 will do with inference.py.

6. Evaluation Criteria

Hidden missing values found	Was '?' detected and replaced with np.nan? Did isnull().sum() change before/after?
Feature engineering quality	Are all 5 engineered features present and documented before the pipeline?
Pipeline reuse	Is the same ColumnTransformer architecture used as in A02 — no regression in structure?
No hardcoded scores	Every number in both results tables comes from computed outputs.
RandomizedSearchCV usage	Is n_iter set? Is refit=True? Is StratifiedKFold passed explicitly?
Tuning justification	Is the choice of search space documented? Is the improvement (or lack thereof) explained?
Artifact verification	Is the round-trip assert present and passing?

inference.py correctness	Does predict() work standalone? Does validate_input() raise ValueError on bad input?
inference.py A04 readiness	Does the output match the dict contract? Would the A04 view above work with zero changes?
Self-directed analysis	≥3 original charts, ≥5 specific findings with numbers. Not generic.
Written summary	Does it mention what A04 will do with the inference script?

7. Rules & Constraints

Allowed

pandas, numpy, matplotlib, seaborn, scipy, scikit-learn (all modules), joblib, pathlib, os.

imbalanced-learn only in the borrowed-technique section.

Prohibited

xgboost, lightgbm, catboost, tensorflow, keras, torch. No AutoML. No deep learning.

No hardcoded metric values anywhere in either results table.

inference.py must not import anything from the notebook or rely on notebook-defined variables.

Carry-forward rules — still tested

- Feature engineering before pipeline. Checked explicitly.
- Coverage assert (all columns covered by numeric + categorical lists).
- All CV scores computed, not hardcoded.
- StratifiedKFold passed explicitly to cross_validate.
- Self-directed analysis: ≥3 charts, ≥5 specific findings.
- Real notebook URLs in References. No invented titles.

8. Hints & Traps

1. The '?' replacement must happen on the raw df BEFORE you split into X and y. If you split first and replace after, y will not be affected but the replacement on X might be applied inconsistently. Do it in one place: df.replace('?', np.nan, inplace=True).

2. income column values have a leading space: ' >50K' and ' <=50K'. If you encode before stripping, your target column will map incorrectly — 'No' rows will become NaN. Always strip all object columns before any encoding or mapping.
3. The double-underscore notation in param_distributions uses the step names you gave in the Pipeline. If you named your classifier step 'clf' instead of 'classifier', the key is clf__n_estimators, not classifier__n_estimators. Check your step names with pipeline.named_steps.keys() before writing the param dict.
4. RandomizedSearchCV with refit=True automatically retrains the best configuration on the full X_train after search. search.best_estimator_ is that refitted pipeline. Do not call .fit() on it again — it is already fitted.
5. joblib is preferable to pickle for sklearn Pipelines because it handles large numpy arrays more efficiently. The artifact includes the fitted StandardScaler means/variances and OneHotEncoder category lists — those are numpy arrays internally. Use joblib, not pickle.
6. In inference.py, feature engineering must be repeated in validate_input(). The pipeline only handles imputation, scaling, and encoding — it does not know about your engineered features. If you created ChargesPerMonth in the notebook, you must compute it again in validate_input() before passing the DataFrame to pipeline.predict().
7. Test inference.py by running it as a plain Python script from the command line, not just as a notebook cell. If it has any notebook dependencies it will fail outside the notebook. That failure is a bug you must fix before submitting.

9. Submission Checklist

Day 1 Checklist

☐ '?' detected via df.isin(['?']).sum() before any replacement
☐ df.replace('?', np.nan) applied; isnull().sum() shows real missing counts
☐ All object columns stripped of whitespace before encoding
☐ income column mapped to 0/1 after stripping
☐ Class distribution printed and visualised
☐ All 5 engineered features created before the pipeline: age_group, hours_category, capital_net, is_married, education_level
☐ Each engineered feature documented in markdown
☐ Column coverage assert present and passing
☐ Baseline pipeline fitted; classification_report and AUC printed
☐ All 4 models evaluated with computed CV scores in results table

Day 2 Checklist

☐ RandomizedSearchCV defined with n_iter=30, refit=True, explicit StratifiedKFold
☐ search.best_params_ and search.best_score_ printed
☐ Tuning comparison table: baseline CV AUC vs tuned CV AUC vs validation AUC — all computed
☐ Tuning result (improvement or not) explained in markdown
☐ artifacts/income_pipeline.pkl saved via joblib.dump()
☐ Round-trip assert passing (original_preds == loaded_preds).all()

- ☐ File size printed
- ☐ inference.py exists as a standalone .py file (not a notebook cell)
- ☐ predict() tested with ≥3 sample inputs from test set
- ☐ predict() tested with a missing-field error case — ValueError raised with message
- ☐ Latency of a single predict() call printed
- ☐ Self-directed analysis: ≥3 charts, ≥5 specific findings
- ☐ 3 real Kaggle notebook URLs in ## References
- ☐ Written summary mentions A04 bridge

Final Submission Checklist

- ☐ Notebook runs top-to-bottom after Restart & Run All — no errors
- ☐ All cell outputs visible
- ☐ No prohibited libraries imported
- ☐ No hardcoded CV or metric values in any results table
- ☐ inference.py runs as a standalone Python script outside the notebook
- ☐ artifacts/income_pipeline.pkl is included in the submission zip

A model that can't be called is a model that doesn't exist. Ship it.